TDP-Reasoner: Timing-Dependent Plasticity Along the Recursion

Anonymous Author(s)
Affiliation
email

Abstract

Small recursive models trained from scratch are the strongest non-LLM approach to puzzle benchmarks such as Sudoku-Extreme and ARC-AGI. These models, HRM and TRM among them, re-apply a Transformer block many times. Seen as a modern Hopfield network, each step's attention is a Hebbian memory of the current state, so recursion reinforces the model's current answer, and accuracy falls once it runs past its training horizon. We propose TDP-Reasoner (timing-dependent plasticity reasoner), which keeps a fast-weight memory along the recursion itself: keys and values from different steps are paired under a learned two-sided timing window, borrowed from spike-timing-dependent plasticity, so the memory tracks how the state moves rather than where it is. Demonstrations of a task reach the query only through a per-task linear rule fitted in context, so unseen tasks need no retraining. Trained for 256 recursion blocks and run for 10,000, it raises held-out Sudoku-Extreme accuracy from 59.8% to 82.0% and never loses a solved puzzle. On unseen ARC-AGI-1 tasks with small grids it matches the public TRM checkpoint (43.2% vs. 45.5%) after about 3% of its training; at full scale it reaches 18.8% pass@2 on ARC-AGI-1 (TRM: 41.8% under the same evaluation) and 0% on ARC-AGI-2.

1 Introduction

[To be written.]

2 Related work

[To be written.]

3 TDP-Reasoner

We start from recurrent linear attention, add a memory across the recursion (the Sudoku model), and then add a rule head that lets the model read the demonstrations of an ARC task.

3.1 Starting point: recurrent linear attention

One block with shared weights is applied at steps $n = 1, 2, \ldots$ of the recursion. At step n it reads

$$\text{read}_n = \left(\sum_\tau v_{n,\tau} k_{n,\tau}^\top \right) q_n, \tag{1}$$

linear attention over the tokens τ with unit-length queries and keys (QK-norm), followed by the usual residual, RMSNorm and feed-forward layer (Appendix A). Within a step this is ordinary attention. Across steps nothing is kept except the state itself.

3.2 Traces and a timing window

TDP-Reasoner adds a fast weight W that is carried from step to step:

$$\text{read}_n = \left(\sum_\tau v_{n,\tau} k_{n,\tau}^\top + W_n \right) q_n. \tag{2}$$

W_n holds every pair of a value and a key of the same token taken at two steps $t, s \le n$, weighted by a window of their time difference:

$$W_n = \sum_\tau \sum_{s,t \le n} \rho_{n-\max(s,t)} \, L(t-s) \, v_{t,\tau} k_{s,\tau}^\top. \tag{3}$$

With the key as presynaptic and the value as postsynaptic activity, this is the pair-based STDP integral $\Delta w = \sum W(t_{\text{post}} - t_{\text{pre}}) \, \text{post} \cdot \text{pre}$: pairs in which the key came first ($t > s$) fall on one side of the window and pairs in which the value came first on the other, so L can strengthen one order and weaken the other. The factor $\rho_a = \eta(1-\eta)^a$ slowly forgets old pairs and keeps W bounded. Equation (3) is computed online with two decaying traces per token, one of keys and one of values, at the cost of two matrix products per step; L is a sum of a few damped complex exponentials whose poles and coefficients are learned (Appendix B; Section 4 proves that the online computation is exact). *[Figure: windows learned on Sudoku.]*

This is the Sudoku model: recurrent linear attention with QK-norm, plus traces and a timing window.

3.3 Reading demonstrations

For ARC every example, an input grid with its output grid, is a *lane* that the block processes on its own; the output grid is hidden in every lane, demonstration or query. The lanes of one task form a *group*. After every block a rule head fits, for each lane, a linear map Θ from cell features ϕ to colours over the *other* demonstrations of its group, one lasso step at a time:

$$\Theta \leftarrow \text{soft}_\lambda \big(\Theta - (\Theta C - B) \big), \qquad r = \Theta \phi, \tag{4}$$

where C and B average $\phi\phi^\top$ and $y\phi^\top$ (y the one-hot colour) over those demonstrations' output cells, and soft_λ is soft-thresholding. The prediction r is added to the next block's input on the output canvas. A query never enters C or B, so it is answered from the demonstrations alone, with no puzzle embedding and no test-time training. As in the Tolman–Eichenbaum machine, the features are shared structure and Θ binds them to content for one task; a few position frames in the rule head retrieve colours by location (Appendix C).

3.4 Training and inference

A segment runs a fixed number of blocks and backpropagates through the last few; the state, traces, W and Θ are carried to the next segment without gradient, with a loss after every segment (TRM's deep supervision without halting) and TRM's stablemax cross-entropy. At inference, as in TRM, each task is run in several augmented versions and the answers are voted; ties are broken by how well each version predicts its own demonstrations (Appendix A).

4 Theory

[To be written: the writes equal the pair-based STDP integral; the window sees only the lagged cross-correlation of activities; recursion supplies temporal order.]

5 Experiments

[To be written.]

6 Limitations and future work

[To be written.]

A The block, segments and inference

A lane's state $h \in \mathbb{R}^{T\times d}$ starts from a fixed vector. At step n, with x the lane's tokens and r_{n-1} the rule head's prediction on the output canvas,

$$u_n = h_{n-1} + \sqrt{d}\, e_n, \qquad e_n = \mathrm{Embed}(x) + \begin{bmatrix} 0 \\ W_{\mathrm{fb}}\, r_{n-1} \end{bmatrix},$$

$$\tilde{q}_n = \mathrm{RoPE}\big(q_n/\|q_n\|\big), \quad \tilde{k}_n = \mathrm{RoPE}\big(k_n/\|k_n\|\big), \quad [q_n;k_n;v_n] = [W_q;W_k;W_v]\,u_n \quad \text{(per head)},$$

$$h'_n = \mathrm{RMSNorm}\left(u_n + W_O\,\mathrm{read}_n\right), \qquad h_n = \mathrm{RMSNorm}\left(h'_n + \mathrm{FFN}(h'_n)\right),$$

with read_n from (2) computed per head, FFN a SwiGLU layer and parameter-free RMSNorm. RoPE uses either fixed one-dimensional frequencies over the token index or learned angles on canvas rows and columns (Section 5 says which run uses which). An ARC lane stacks the input and output canvases ($C \times C$ each, $C = 30$) into $T = 2C^2$ tokens over {unknown, pad, colours 0–9, end of grid}.

[Segments: blocks per segment and backpropagated blocks per run; 16 segments per lane in full-scale runs, one in the small-grid harness. Inference: versions, inverse transforms, voting, tie-break by demonstration consistency.]

B Computing the memory online

For every head and token, each band $r = 1,\dots,R$ keeps a key trace and a value trace over the steps:

$$E^K_r(n) = z_r E^K_r(n-1) + \gamma_r \tilde{k}_n, \qquad D^V_r(n) = z_r E^V_r(n-1), \qquad E^V_r(n) = D^V_r(n) + \gamma_r v_n,$$

$$P_n = \sum_{\tau} \sum_{r=1}^{R} \mathrm{Re}\Big(a_r v_n E^K_r(n)^\top + b_r D^V_r(n)\tilde{k}_n^\top\Big), \qquad W_n = (1-\eta)\,W_{n-1} + \eta\, P_n,$$

with learned poles $z_r = \exp(-e^{\nu_r} + i\theta_r)$, $\gamma_r = \sqrt{1-|z_r|^2}$ (the parameterisation of linear recurrent units with γ_r tied to the pole), learned complex coefficients a_r, b_r and a learned rate $\eta \le 1/8$, all per head. The window of (3) is

$$L(\Delta) = \sum_r \gamma_r\, \mathrm{Re}\left(a_r z_r^{\Delta}\right)\ (\Delta \ge 0), \qquad L(\Delta) = \sum_r \gamma_r\, \mathrm{Re}\left(b_r z_r^{|\Delta|}\right)\ (\Delta < 0),$$

which contains the usual exponential STDP window ($R = 1$, $\theta = 0$, $a > 0$, $b < 0$). We use $R = 4$ bands with initial periods of 2, 6, 20 and 64 steps; a_r, b_r start small and random. The bands are summed before the token sums, so a step costs two matrix products for any R. Traces and W start at zero for a new lane.

C Rule head

Features are $\phi(o) = W_\phi h(o)/\|W_\phi h(o)\| \in \mathbb{R}^p$ on the output-canvas cells o. For a lane ℓ, with n_ℓ the number of labelled output cells in the other demonstration lanes ℓ' of its group,

$$C_\ell = \frac{1}{n_\ell} \sum_{\ell' \neq \ell} \sum_o \phi_{\ell'}(o)\, \phi_{\ell'}(o)^\top, \qquad B_\ell = \frac{1}{n_\ell} \sum_{\ell' \neq \ell} \sum_o y_{\ell'}(o)\, \phi_{\ell'}(o)^\top,$$

and (4) is one proximal step on $\frac{1}{2}\operatorname{tr}(\Theta C_\ell \Theta^\top) - \operatorname{tr}(\Theta B_\ell^\top) + \lambda \|\Theta\|_1$, valid at unit step because $\lambda_{\max}(C_\ell) \leq \operatorname{tr} C_\ell = 1$. The prediction has a frame term, $r(o) = \Theta\phi(o) + \sum_{k=0}^{K} \beta_k \kappa_k(o)$: κ_0 is the colour histogram of the lane's input grid and κ_k ($k \geq 1$) a softmax read of the input grid's colour cells from cell o with learned rotary frequencies on canvas coordinates ($K = 8$, scaled by $1/\sqrt{K+1}$); β belongs to the same lasso. r enters the next block through W_{fb}, initialised at zero. The logits are classifier$(h(o)) + \tau\, r(o)$, or an output FFN applied to $h(o) + \sqrt{d}\, W_{\text{fb}} r(o)$ (Section 5).